

Trace-Driven Microservice Co-location for Efficient Resource Use in Constrained Kubernetes Clusters

TARO JOHO^{1,†1,a)} HANAKO SHORI^{2,b)} JIRO GAKKAI¹

Received: March 4, 2016, Accepted: August 1, 2016

Abstract: Resource-constrained Kubernetes clusters often place replicas of different microservices on the same worker node, a practice referred to as microservice co-location. Poor co-location can cause contention because CPU demand alone does not capture differences in service invocation frequency. We propose a trace-driven method that combines measured CPU demand with trace-derived invocation factors to estimate workload-dependent node pressure and guide placement. Under sustained high load near the system’s capacity limit, the proposed method reduced mean successful-request latency by 14.6% relative to the default scheduler. Under severe overload, it increased the success rate from 18.0% to 51.0%, demonstrating greater overload resilience without additional resources.

Keywords: Microservices, Kubernetes, Cloud-edge computing, Trace-driven scheduling, Resource reconfiguration

1. Introduction

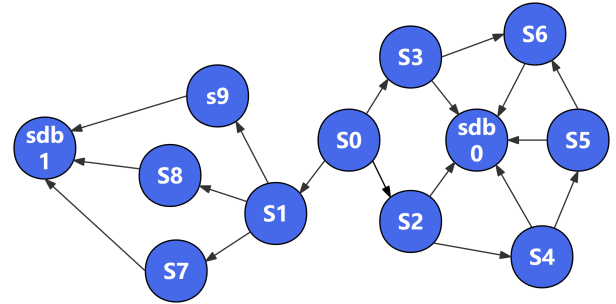
Kubernetes clusters often operate with finite worker-node capacity, particularly in resource-constrained cloud-to-edge environments [1], and microservice applications share computing resources to improve overall cluster utilization [2]. The practice of placing replicas of different microservices on the same worker node is referred to as microservice co-location [3]. However, inappropriate service combinations can concentrate workload pressure, cause resource contention, and lead to service-level objective (SLO) violations [4]. Effective co-location therefore requires identifying service combinations that use limited node capacity efficiently without concentrating workload-dependent pressure.

However, resource demand alone is insufficient for evaluating co-location risk because services participate differently in request processing. Figure 1b summarizes 652 application–service pairs from 40 Alibaba-derived applications provided by μ Bench [5]. For each pair, trace coverage is defined as the percentage of traces from the corresponding application in which the service appears at least once. Among these services, 61.0% appear in no more than 20% of traces for each application, whereas 20.6% appear in more than 80%. This distribution demonstrates uneven service participation: many services are invoked only by a small fraction of requests, while a smaller group participates in most requests. Similar differences have also been observed in production workloads [6], [7]. Because service demand is incurred only when the service is invoked, workload-dependent pressure must be evaluated using both measured CPU demand and trace-derived invoca-

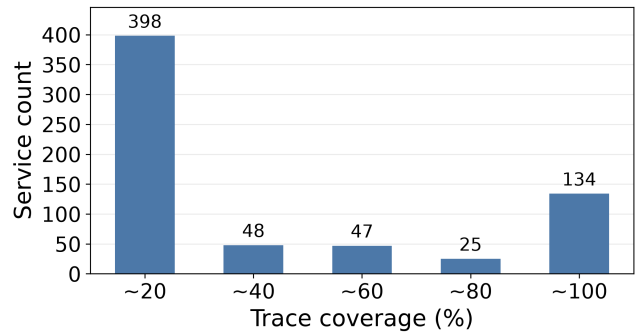
tion characteristics.

Based on this principle, this paper makes the following contributions:

- *Invocation-aware pressure characterization:* We combine measured service-level CPU demand with a trace-derived invocation factor that represents the average number of service invocations generated by one entry request. This character-



(a) Example service call graph.



(b) Service distribution by trace coverage.

Fig. 1: Call structure and service participation in the Alibaba-derived μ Bench workloads.

¹ Information Processing Society of Japan, Chiyoda, Tokyo 101-0062, Japan

² Johoshori University, Chiyoda, Tokyo 101-0062, Japan

^{†1} Presently with Johoshori University

^{a)} joho.taro@ipsj.or.jp

^{b)} shori.hanako@johosyori-u.ac.jp

ization distinguishes observed processing pressure from the request-level invocation behavior that produces it.

- *Prospective fixed-allocation evaluation:* Without changing replica counts or resource allocations, we aggregate service pressure relative to heterogeneous node capacity and evaluate not-yet-deployed candidate mappings across projected workload levels. Candidates with greater node-overload risk are ranked lower.
- *Trace-driven placement workflow:* We implement an automated workflow that collects distributed traces and CPU metrics from an earlier workload window, generates a lower-risk service-to-node mapping, and applies it to a Kubernetes cluster through service rescheduling.

Together, these contributions provide a trace-driven approach for using limited cluster capacity more effectively. Rather than adjusting co-location only after contention appears, the approach uses previously observed request behavior to screen alternative placements before they are deployed under increasing workload.

2. Related Work

This section reviews prior work relevant to trace-driven microservice co-location. We organize the discussion around resource-aware placement, interference-aware co-location, and trace-driven resource management, and clarify how our method differs from these approaches.

2.1 Resource-aware Scheduling

Resource-aware scheduling in Kubernetes places Pods by filtering nodes that violate resource requirements or placement constraints and then scoring the remaining feasible nodes to select a target [8]. Existing studies demonstrate that placement, capacity, and load-balancing requirements can be formulated as global constraint optimization [9], while global monitoring and real-time resource telemetry can further improve load-aware rescheduling [10], [11]. However, placement feasibility and aggregate resource balance alone do not determine whether the workload-dependent pressures of a particular combination of microservices can safely share a node, especially when dedicated or peak-provisioned capacity for every replica is impractical in resource-constrained clusters. The scheduling problem must therefore extend from deciding where each individual Pod can run to deciding which services should be co-located under limited capacity, motivating the co-location method developed in this work.

2.2 Microservice Co-location

Microservice co-location improves the use of limited cluster capacity by allowing multiple services to share underutilized resources on the same node. Heracles demonstrates that latency-critical and best-effort workloads can safely share CPU, memory, cache, and network resources [12], while FIRM shows that resource sharing among microservices can also introduce contention requiring runtime diagnosis and reprovisioning [2]. Interference-aware methods such as iPlace and PISM therefore select service combinations according to resource competition characteristics or composition-specific interference scores [13], [14]. However, a combination that appears safe under current

resource measurements may become unsafe as request volume or request composition changes, because the demand imposed by each service is workload-dependent. Co-location decisions therefore cannot rely only on the current resource state; they must also anticipate how the pressure of co-located services will change under future workloads, motivating prediction-based resource management.

2.3 Prediction-based Resource Management

Prediction-based resource management anticipates future performance from current observations, but its usefulness for placement depends on representing inter-service dependencies. Sinan predicts tail latency and QoS violation probability under candidate CPU allocations [15], while C-Koordinator predicts hardware-level interference from node-, Pod-, and application-level metrics [4]. However, performance problems in microservice applications are not isolated: Sage shows that service dependencies introduce backpressure and cascading QoS violations [16], while MicroHECL analyzes anomalies that propagate along service call chains and ultimately affect application-level availability [17]. Consequently, predictions based primarily on resource metrics may forecast future latency or interference but omit the request-path context needed to determine how workload pressure originates and propagates across services. This limitation motivates the use of distributed traces to introduce service invocation paths into proactive placement decisions.

2.4 Trace-driven Placement

Distributed traces provide request-path information that resource measurements alone do not expose, including which services are invoked and how often they are executed by an entry request. Existing studies use this information for several management tasks. MicroRCA combines trace-derived dependency graphs with resource metrics for performance diagnosis [18], Colarusso et al. construct workload-dependent graphs to co-locate strongly interacting services and improve communication locality [19], and Morphis predicts recurring call-graph patterns to support SLO-aware CPU provisioning [7].

Co-location has also been optimized using runtime resource and interference information. REMaP adapts the current placement according to observed communication affinity and resource-usage history, avoiding configurations that separate strongly interacting services or concentrate high resource usage [20]. iPlace instead groups microservices with diverse resource-competition characteristics to reduce interference among co-located services [21]. These methods derive placement decisions primarily from interactions, resource usage, or contention characteristics observed under the current operating configuration. In contrast, our method uses CPU demand and invocation behavior collected during an earlier workload window to evaluate service-to-node mappings that have not yet been deployed. Without changing replica counts or resource allocations, it projects their node pressure across multiple workload levels and selects the placement with lower projected overload risk.

3. Trace-Driven Co-location Scheduler

The proposed framework interprets a distributed trace not only as a service dependency graph, but also as a temporal record of how computing resources are occupied during request processing. It first identifies the execution intervals and invocation multiplicities of individual services from traces, and then combines this information with measured CPU demand to estimate how service pressure changes as request concurrency increases. Based on these estimates, the framework generates candidate co-locations and evaluates whether the remaining capacity of each node can safely accommodate the resulting aggregate pressure. Figure 3 illustrates the overall workflow, consisting of trace-derived window characterization, workload-dependent pressure estimation, candidate placement generation, and load-growth-aware filtering.

3.1 Co-location Windows

Distributed traces reveal potential temporal headroom between service executions. As illustrated in Fig. 2, the CPU-active phase of S1 does not fully overlap with the main execution phases of S2, S3, and S4. Placing services with such complementary execution intervals on the same node could improve cluster utilization without imposing their full CPU demands simultaneously. Prior work provides evidence that this trace-visible headroom can be exploited.

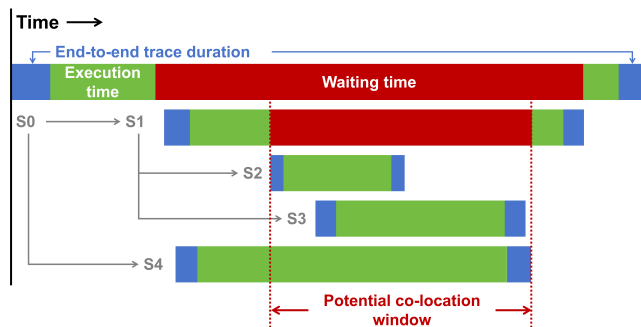


Fig. 2: Illustration of a potential co-location window within a request trace.

Trace-visible idle intervals may indicate potential co-location opportunities. Zhu et al. use per-request execution footprints and critical-path analysis to identify such intervals and reclaim idle resources for co-scheduling [22]. However, an interval observed in an individual trace cannot be treated as fixed node capacity because concurrency and execution blocking change its timing and availability [23]. Therefore, these temporal opportunities must be evaluated across the workload rather than matched directly between individual requests.

This does not necessarily mean that the underlying co-location headroom disappears. Instead, uneven service invocation frequency [7] and different request arrival times distribute it across fragmented temporal intervals. Our method abstracts these fragments by normalizing each service's cumulative CPU consumption over the complete observation window as CPU/s. The resulting CPU/s demands are aggregated and compared with node ca-

capacity under increasing request volumes, converting fragmented temporal windows into a resource-based co-location window. The detailed normalization and co-location scoring procedure is presented in Section 4.

3.2 Trace-Driven Framework

The proposed framework performs proactive microservice co-location through a periodic observation and adjustment cycle. As shown in Fig. 3, each cycle consists of telemetry collection, trace-metric evaluation, and deployment-based rescheduling. Separating evaluation from execution allows alternative placements to be examined under projected workloads before any running service is moved. The following sections describe these three stages in their execution order.

3.2.1 Telemetry Collection

Each adjustment cycle begins by collecting application- and resource-level information from the running cluster. Istio sidecars record distributed traces in Jaeger, including request paths and service invocation counts, while Kubelet/cAdvisor and Prometheus provide container-level CPU measurements; the current placement and node capacities are obtained through Kubernetes. These data connect the invocation characteristics of each service with its measured CPU demand and current execution environment. The collected observations are subsequently used to evaluate alternative co-location plans.

3.2.2 Scoring and Evaluation

The framework jointly analyzes trace and metric information to estimate the workload-dependent pressure of each service. Measured CPU demand is combined with trace-derived invocation frequency and projected request volume, after which alternative service-to-node mappings are evaluated against the available capacity of each node. Infeasible candidates are filtered out, while the remaining plans are ranked according to their estimated overload area and resource balance, with the current deployment retained when no candidate provides an improvement. The highest-ranked feasible plan is then passed to the co-location rescheduler.

3.2.3 Deployment-Based Rescheduling

The co-location rescheduler applies the selected plan by updating the placement constraints of only the affected Kubernetes Deployments through the API server. It then triggers the Deployment rolling-update mechanism, allowing Kubernetes controllers to create replacement Pods and the default scheduler to place them on the selected nodes before the old Pods are terminated under the configured readiness and availability policy. Services unaffected by the selected plan continue running without redeployment, reducing unnecessary disruption. The updated placement becomes the cluster state used in the next observation cycle.

4. Methodology

The proposed method uses a staged search that combines baseline-adjusted CPU usage, trace-derived calls per entry request, and configured Pod CPU requests. These parameters represent observed processing demand, uneven service participation, and Kubernetes placement feasibility, respectively. As summarized in Algorithm 1, the framework first generates balanced candidates through randomized greedy search and local refinement,

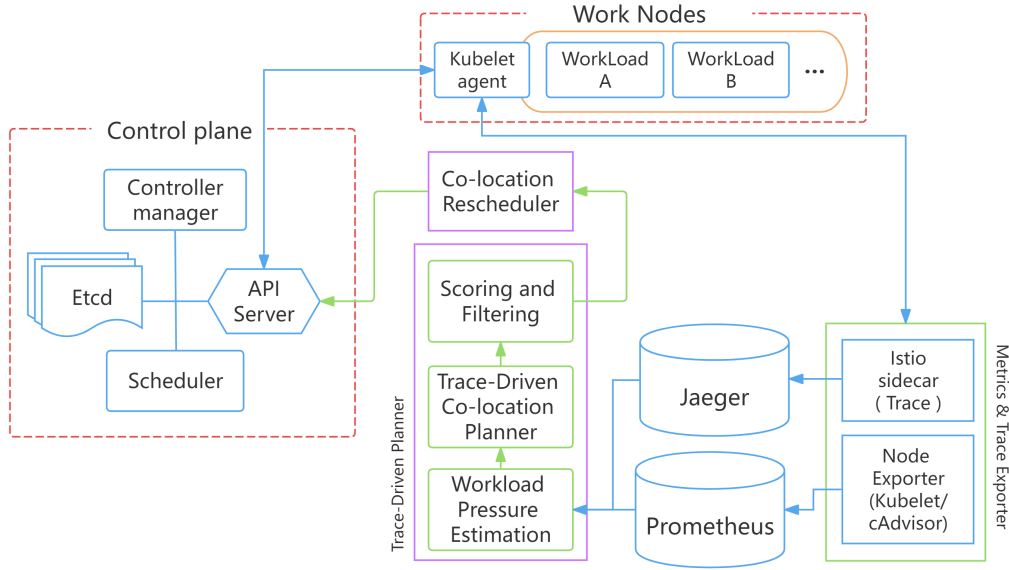


Fig. 3: Architecture of the proposed trace-guided scheduling support method.

Algorithm 1 Candidate placement generation and selection

Require: Replicas P with (u_p, f_p, q_p) , nodes N with capacity C_n , and restarts $M = 300$

Ensure: Five lowest-scoring candidates C_5 and selected placement x^*

- 1: $C \leftarrow \emptyset, C_{\max} \leftarrow \max_{n \in N} C_n$
- 2: $\mathcal{H} \leftarrow$ selected potential hotspot services
- 3: **for** $i = 1$ to M **do**
- 4: Order P by $u_p/C_{\max} + f_p + q_p$ with randomized tie-breaking
- 5: $x \leftarrow$ feasible greedy placement minimizing partial $B(x)$
- 6: **if** x exists **then**
- 7: Refine x through replica relocation and exchange
- 8: $C \leftarrow C \cup \{x\}$
- 9: **end if**
- 10: **end for**
- 11: **for all** $x \in C$ **do**
- 12: Compute global and hotspot overload areas
- 13: Compute overload score $R(x)$
- 14: **end for**
- 15: Rank C lexicographically by $(R(x), B(x))$
- 16: $C_5 \leftarrow$ first five candidates in C
- 17: $x^* \leftarrow$ first candidate in C_5
- 18: **return** C_5, x^*

and then ranks them according to their overload risk under general workload growth and potential hotspot changes. The following subsections describe the input parameters, balance-guided candidate generation, and trace-aware overload selection.

4.1 Baseline-Adjusted CPU Usage

The framework estimates the CPU/s required by a typical Pod from the total CPU usage of its service. For each service, the baseline-adjusted CPU usage of its observed Pods is aggregated and divided by the configured number of replicas:

$$u_s = \frac{1}{r_s} \sum_{p \in \text{Pods}(s)} \max\left(0, \frac{\Delta C_p}{\Delta t} - b_p\right), \quad (1)$$

where ΔC_p is the increase in cumulative CPU time of Pod p

during observation window Δt , b_p is its average idle CPU usage, and r_s is the number of replicas of service s . A Pod-specific baseline is used when available; otherwise, the service's mean per-Pod baseline is adopted. The resulting u_s represents the estimated CPU/s per Pod and is used for candidate placement evaluation.

4.2 Trace-Derived Call Factor

The trace-derived call factor represents the average number of times a service is invoked by one entry request. The trace analyzer counts server-side spans in the Jaeger JSON files. Client-side spans are excluded to avoid double-counting an inter-service call, while repeated server-side spans are retained because they represent separate executions of the destination service.

For service s , the service-level call factor and its estimated per-Pod value are calculated as

$$F_s = \frac{N_s}{N_e}, \quad f_s = \frac{F_s}{r_s}, \quad (2)$$

where N_s is the total number of server-side calls to service s , N_e is the number of observed entry requests, and r_s is the configured number of replicas of service s . Thus, F_s measures the average number of calls to service s generated by one entry request, while f_s estimates the corresponding invocation demand per Pod. A value below one indicates that the service does not participate in every request, whereas a value above one indicates repeated invocation in some request paths. The resulting f_s is used as the trace-derived input for candidate placement evaluation.

4.3 Balance-Guided Candidate Generation

The framework uses the balance score $B(x)$ to guide candidate generation toward placements with lower dispersion in measured CPU pressure, trace-derived invocation demand, and reserved CPU. For each node n , U_n is the aggregate estimated CPU/s of its assigned replicas, F_n is their average per-Pod call factor, and Q_n is their aggregate CPU request. Since the worker nodes have

heterogeneous CPU capacities, both U_n and Q_n are normalized by the node capacity C_n . The relative dispersion of the three indicators is calculated as

$$\begin{aligned} V_U &= \sum_{n \in N} \left(\frac{U_n/C_n - \bar{U}}{\bar{U}} \right)^2, & V_F &= \sum_{n \in N} \left(\frac{F_n - \bar{F}}{\bar{F}} \right)^2, \\ V_Q &= \sum_{n \in N} \left(\frac{Q_n/C_n - \bar{Q}}{\bar{Q}} \right)^2, & B(x) &= V_U + V_F + V_Q. \end{aligned} \quad (3)$$

Here, C_n is the CPU capacity of node n , while $\bar{U}$, $\bar{F}$, and $\bar{Q}$ denote the node means of U_n/C_n , F_n , and Q_n/C_n , respectively. The quantities U_n/C_n and Q_n/C_n are CPU ratios, and F_n is a call-per-request ratio. Therefore, all three inputs and their relative-dispersion measures V_U , V_F , and V_Q are dimensionless. If the mean of an indicator is zero, its dispersion is defined as zero because all of its non-negative node values must also be zero.

The three dispersion terms are combined with equal weight as a heuristic measure of overall balance. A smaller $B(x)$ indicates a lower combined dispersion across the three indicators, but does not imply that every indicator is individually minimized. Placements with $Q_n > C_n$ are rejected because their CPU requests exceed node capacity. The score guides the randomized greedy and local-search procedures toward balanced feasible candidates; it does not guarantee a globally optimal placement. The resulting candidate pool is subsequently evaluated using the overload score described in the next subsection.

The candidate-generation method builds on established principles of combinatorial assignment optimization. Rebalancer demonstrates that large-scale resource-assignment problems with capacity constraints can be efficiently addressed through local search based on object relocation and exchange [24]. Meanwhile, GRASP establishes randomized multi-start construction followed by local improvement as a general method for exploring discrete solution spaces and reducing dependence on a single initial solution. Following these principles, the proposed framework treats $B(x)$ as a domain-specific objective, preserves CPU-request feasibility during relocation and exchange, and explores multiple locally improved placements. Thus, the search structure is grounded in established optimization methods, while the integration of measured CPU/s, trace-derived invocation intensity, and reserved CPU into the balance objective is specific to this work.

4.4 Trace-Aware Overload Scoring

The overload score evaluates each candidate under projected CPU demand. Let λ denote the projected entry-request rate relative to the observed rate. If the mean CPU cost per invocation and the general call structure remain approximately stable, CPU demand increases proportionally with the request rate, following the utilization law [25]. Therefore, λU_n represents the CPU demand offered to node n , rather than realized CPU utilization beyond its capacity.

Two demand scenarios are considered. In the general-growth scenario, $U_n^{(G)}$ is the aggregate measured CPU/s of the replicas assigned to node n . In the hotspot-growth scenario, $U_n^{(H)}$ is obtained from the same placement by additionally scaling the CPU/s

of potential hot services according to the projected change in their call factors. Thus, the hotspot scenario includes both the general request-rate growth represented by λ and the additional invocation growth of selected services. The overload areas and their weighted score are calculated as

$$\begin{aligned} A_k(x) &= \sum_{n \in N} \int_{\lambda_{\min}}^{\lambda_{\max}} \left[\max \left(0, \frac{\lambda U_n^{(k)}}{C_n} - 1 \right) \right]^2 d\lambda, & k &\in G, H, \\ R(x) &= (1 - \omega)A_G(x) + \omega A_H(x), & 0 \leq \omega \leq 1. \end{aligned} \quad (4)$$

Here, C_n is the CPU capacity of node n . The term inside the max function is the normalized CPU demand exceeding node capacity. Demand below capacity incurs no penalty, while squaring the excess gives greater weight to severe overload. For the same total normalized excess, the sum of squares is also larger when overload is concentrated on fewer nodes. The score therefore reflects both the amount and the distribution of projected overload across heterogeneous nodes.

The weighting factor ω controls the relative importance of the two scenarios, whereas a larger value places greater emphasis on hotspot growth and a smaller value emphasizes proportional growth under the observed invocation structure. The two areas remain separate evaluation criteria; their weighted combination does not imply that the loads from the two scenarios occur simultaneously. The value of ω can therefore be configured according to the expected workload uncertainty and deployment objective. In this evaluation, ω is set to 0.5 to assign equal importance to the two scenarios. Candidates are ranked by $R(x)$, with $B(x)$ used to break ties. The selected placement is the lowest-scoring candidate in the generated pool; global optimality is not claimed.

5. Evaluation

5.1 Experimental Setup

We constructed an emulated cloud-edge testbed using KVM virtual machines on a physical server equipped with two Intel Xeon E5-2650 v2 processors and approximately 60 GiB of memory. The testbed runs MicroK8s and consists of one control/client node and four worker nodes, as summarized in Table 1.

Table 1: Configuration of the experimental cluster.

Node	Role	vCPU	Memory (GiB)	Disk (GiB)
client-0	Control/client	2	4	40
cloud-0	Cloud worker	8	16	50
edge-1	Edge worker	4	8	30
edge-2	Edge worker	2	4	30
edge-3	Edge worker	2	4	30

The application Pods are deployed only on the four worker nodes, while client-0 hosts the control-plane functions and generates the test workloads. The heterogeneous worker capacities, with a CPU ratio of 8 : 4 : 2 : 2, represent one resource-rich cloud node and three resource-constrained edge nodes. Istio collects distributed traces, while Prometheus records Pod- and node-level CPU metrics.

Table 2: Controlled invocation patterns used in the evaluation.

Pattern	Common operations	Business operation
Trace 01	s2×2, s11×2	s8×2
Trace 02	s2×2, s11×2	–
Trace 03	s2×2, s11×2	s10×1
Trace 04	s2×2, s11×2	s13×1

5.2 Microservice Application Workload

The evaluation uses a branching microservice application to represent request-dependent service participation. As shown in Figure 4, s0 receives entry requests, while s1–s14 form the internal services and their possible dependencies. For example, s7 invokes the branches rooted at s3 and s11, while s3 connects to s2, s8, s10, s12, and s13; each request activates only a subset of this topology. This common structure supports controlled comparison across different trace patterns.

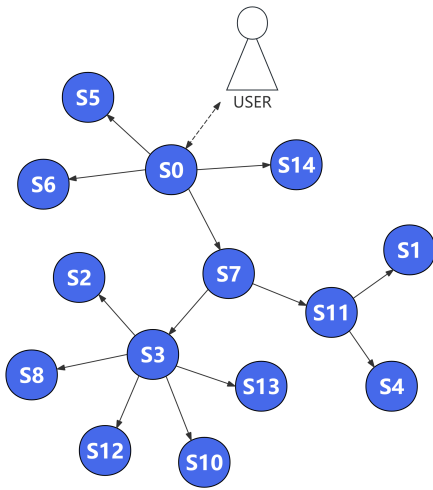


Fig. 4: Microservice topology used in the evaluation.

Four recurring trace patterns introduce controlled differences in invocation frequency and resource demand. As summarized in Table 2, the lightweight services s2 and s11 are invoked twice in every pattern, whereas the resource-intensive services s8, s10, and s13 appear only in specific patterns. Their relative per-invocation workload intensities are set to (1:1:1.5:2:2), separating persistent lightweight pressure from occasional resource-intensive pressure. This design allows us to evaluate whether a placement method captures both resource demand and trace-dependent invocation tendency.

5.3 Comparison Design

The evaluation uses three placement methods to determine whether trace-derived information provides benefits beyond resource-aware re-placement. Comparing Proposed only with Default would reveal the overall improvement but could not distinguish the contribution of trace information from that of CPU/s-aware load balancing. We therefore introduce Greedy as a resource-only baseline between Default and Proposed to evaluate the additional effect of trace-aware overload selection. The three methods are defined as follows:

- *Default* retains the original placement produced for the de-

ployed Kubernetes application. It performs no offline candidate generation and does not use measured CPU/s or trace-derived invocation information. This method represents the operational baseline under the same resource budget and workload.

- *Greedy* serves as the resource-aware baseline, following the established use of greedy strategies for container scheduling in cloud-edge environments [26]. It selects the placement with the smallest dispersion of capacity-normalized node CPU pressure using measured service CPU/s. It therefore balances the observed CPU load but does not consider trace-derived call factors or projected overload.
- *Proposed* uses the same CPU/s measurements, feasibility constraints, candidate-generation framework, and search budget as Greedy. It additionally uses trace call factors to project node pressure at multiple workload levels and selects the candidate with the smallest robust overload score, as described in Section 4. This enables it to avoid placements that may become overloaded as request volume increases.

Table 3: Pairwise comparisons and their evaluation targets.

Comparison	Evaluated effect
Default vs. Greedy	Benefit of CPU/s-aware placement
Greedy vs. Proposed	Additional benefit of trace-aware overload selection
Default vs. Proposed	Overall benefit of the complete workflow

These comparisons assign a distinct evaluation role to each transition in the placement workflow. As summarized in Table 3, Default versus Greedy measures CPU/s-aware placement, Greedy versus Proposed measures the additional benefit of trace-aware overload selection, and Default versus Proposed measures the overall benefit. Because candidate generation is randomized, Greedy versus Proposed compares the complete procedures under the same search budget rather than using identical candidate sets. Together, these comparisons quantify the benefits of resource awareness, trace awareness, and the complete proposed method.

6. Results

6.1 Request success rate

We evaluate how effectively each co-location uses the fixed cluster capacity by increasing request concurrency from 15 to 50 and measuring end-to-end success rate. All methods use identical resources, replica counts, workload composition, and request sequences, so differences reflect their ability to sustain increasing demand. As shown in Figure 6, all methods achieve approximately 97% success at concurrency 15, while Proposed reaches 86% at concurrency 35 and 51% at concurrency 50, exceeding Greedy and Default by 23 and 33 percentage points at the highest load. This slower degradation indicates that trace-aware overload selection delays saturation and sustains more requests under the same resource budget.

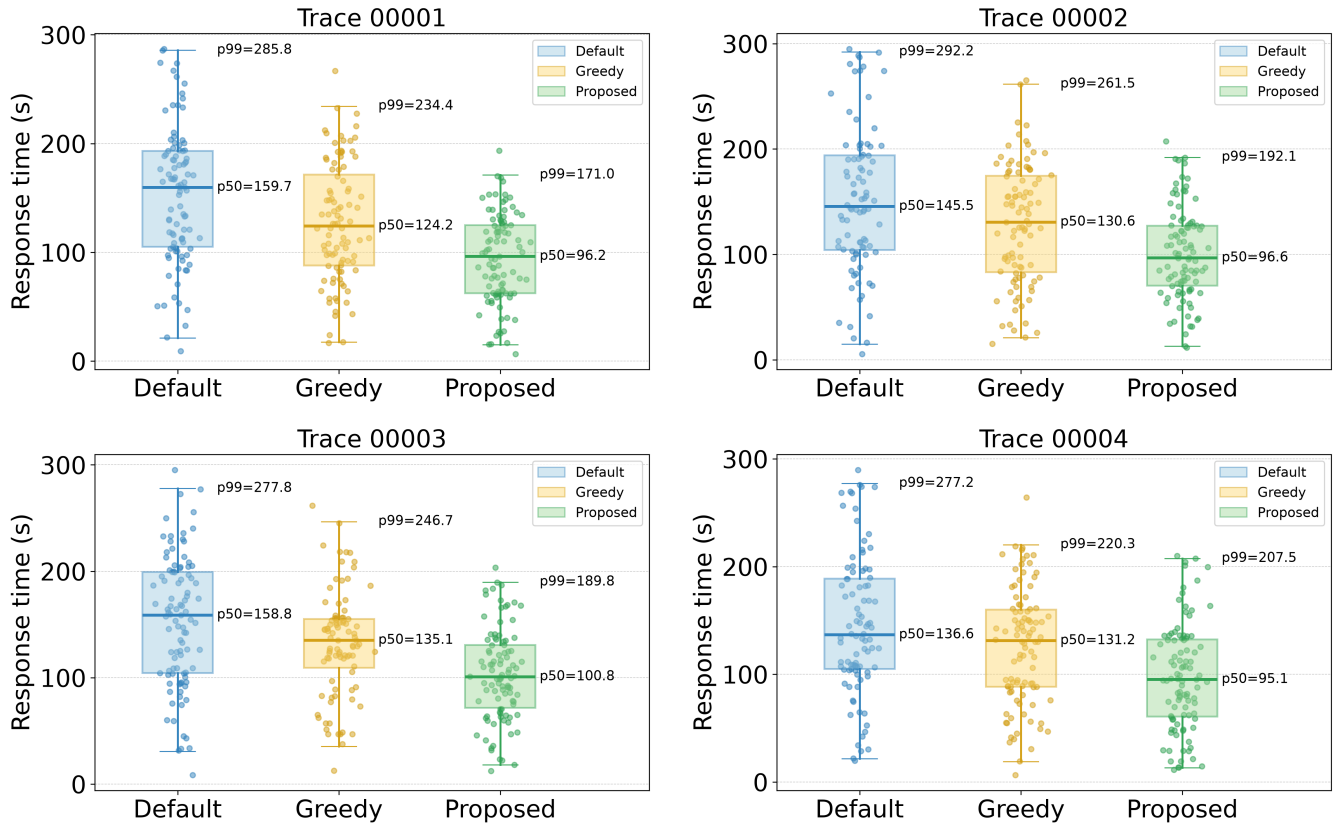


Fig. 5: Successful-request response-time distributions for the four trace patterns at concurrency 20.

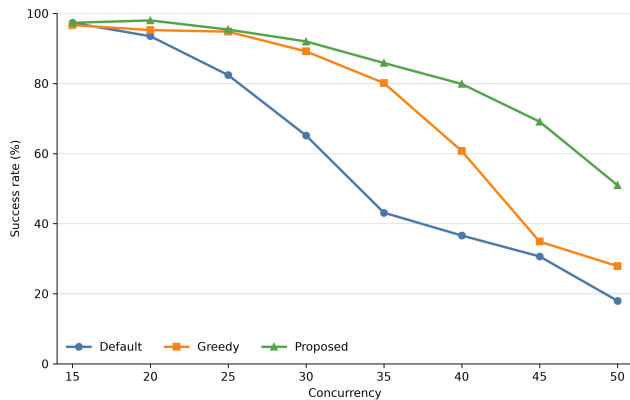


Fig. 6: Request success rate under increasing concurrency.

6.2 SLO Performance

The concurrency sweep demonstrates overall overload resilience, and we further examine the SLO performance of Proposed near cluster capacity. Concurrency 20 is selected because all methods still maintain high request success, enabling a reliable latency comparison with limited survivor bias. As shown in Table ?? and Figure 5, Proposed reduces P50 to 98.7,s, which is 31.5,s (24.2%) lower than Greedy and 50.2,s (33.7%) lower than Default; it also reduces P99 to 203.6,s, improvements of 57.8,s (22.1%) and 85.5,s (29.6%), respectively. These reductions, observed across all trace patterns, show that trace-aware overload selection substantially improves latency-related SLO performance under high resource pressure.

Table 4: Aggregated results at concurrency 20.

Method	Success (%)	Errors	Mean (s)	P50 / P99 (s)
Default	93.5	26	150.7	148.9 / 289.1
Greedy	95.3	19	128.4	130.2 / 261.4
Proposed	98.0	8	98.9	98.7 / 203.6

6.3 Discussion

The results show that Proposed maintains higher request success as concurrency increases and reduces both P50 and P99 latency near cluster capacity. This improvement is consistent across all four trace patterns, indicating that combining measured CPU/s with trace-derived invocation factors helps avoid co-locating services whose accumulated pressure becomes harmful under increasing demand. In contrast, CPU/s balancing improves performance but cannot fully capture invocation-dependent contention. Overall, trace-aware overload selection provides more stable performance under a fixed resource budget.

The current method derives placement decisions offline from previously observed traces and therefore cannot guarantee effectiveness when invocation behavior departs substantially from the observed distribution. This may occur when an unseen trace invokes services in an unexpected pattern, simultaneously activates many services, or invokes an infrequently used resource-intensive service. A similar limitation arises under an abrupt workload-composition shift: for example, one previously observed trace type may increase by more than an order of magnitude while the other trace types disappear. Although the trace pattern itself is known, its resulting service pressures may differ substantially from those represented by the aggregated call factors and the eval-

uated hotspot scenarios. Future work will extend the framework to runtime operation by continuously detecting changes in trace composition, updating call factors, and revising placement decisions accordingly.

Acknowledgments

We wrote this article based on the guideline for A4 landscape layout. We are grateful to Prof. Hiroshi Nakashima from Kyoto University, for his valuable comments on making a class-file, and his consent to usage of BiBTeX files. We are also very thankful to the editorial committee for their contributions in writing the guideline for the A4 landscape layout.

References

- [1] Marchese, A. and Tomarchio, O.: Enhancing the Kubernetes Platform with a Load-Aware Orchestration Strategy, *SN Computer Science*, Vol. 6, p. 224 (online), DOI: 10.1007/s42979-025-03712-z (2025).
- [2] Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T. and Iyer, R. K.: FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices, *14th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, pp. 805–825 (2020).
- [3] Cao, L. and Sharma, P.: Co-Locating Containerized Workload Using Service Mesh Telemetry, *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, ACM, pp. 168–174 (online), DOI: 10.1145/3485983.3494867 (2021).
- [4] Song, S., Xu, M., Zhang, Z., Gao, C., Zeng, F., Ding, Y., Ye, K. and Xu, C.: C-Koordinator: Interference-Aware Management for Large-Scale and Co-Located Microservice Clusters, *Software: Practice and Experience*, Vol. 56, No. 5, pp. 506–524 (online), DOI: 10.1002/spe.70059 (2026).
- [5] Detti, A., Funari, L. and Petrucci, L.: μ Bench: An Open-Source Factory of Benchmark Microservice Applications, *IEEE Transactions on Parallel and Distributed Systems*, Vol. 34, No. 3, pp. 968–980 (online), DOI: 10.1109/TPDS.2023.3236447 (2023).
- [6] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J. and Xu, C.: Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis, *Proceedings of the ACM Symposium on Cloud Computing*, ACM, pp. 412–426 (online), DOI: 10.1145/3472883.3487003 (2021).
- [7] Tang, Y., Zhao, H., Lu, C., Zhang, Y., Chow, K., Deng, S. and Shi, R.: Morphis: SLO-Aware Resource Scheduling for Microservices with Time-Varying Call Graphs, arXiv preprint arXiv:2602.01044 (2026), Version 2.
- [8] Kubernetes: Kubernetes Scheduler, <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/> (2024). Accessed: 2026-05-14.
- [9] Suresh, L., Loff, J., Kalim, F., Jyothi, S. A., Narodytska, N., Ryzhyk, L., Gamage, S., Oki, B., Jain, P. and Gasch, M.: Building Scalable and Flexible Cluster Managers Using Declarative Programming, *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, USENIX Association, pp. 827–844 (2020).
- [10] Jian, Z., Xie, X., Fang, Y., Jiang, Y., Lu, Y., Dash, A., Li, T. and Wang, G.: DRS: A Deep Reinforcement Learning Enhanced Kubernetes Scheduler for Microservice-Based System, *Software: Practice and Experience*, Vol. 54, No. 10, pp. 2102–2126 (online), DOI: 10.1002/spe.3284 (2024).
- [11] Marchese, A. and Tomarchio, O.: Enhancing the Kubernetes Platform with a Load-Aware Orchestration Strategy, *SN Computer Science*, Vol. 6, p. 224 (online), DOI: 10.1007/s42979-025-03712-z (2025).
- [12] Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P. and Kozyrakis, C.: Heracles: Improving Resource Efficiency at Scale, *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, ACM, pp. 450–460 (online), DOI: 10.1145/2749469.2749475 (2015).
- [13] Adeppady, M., Giaccone, P., Karl, H. and Chiasserini, C. F.: Reducing Microservices Interference and Deployment Time in Resource-Constrained Cloud Systems, *IEEE Transactions on Network and Service Management*, Vol. 20, No. 3, pp. 3135–3147 (online), DOI: 10.1109/TNSM.2023.3235710 (2023).
- [14] Yang, D., Zheng, K., Qian, S., Hua, Q., Zhang, K., Cao, J. and Xue, G.: Mitigating Interference of Microservices with a Scoring Mechanism in Large-Scale Clusters, *The Journal of Supercomputing*, Vol. 81, No. 1, p. 104 (online), DOI: 10.1007/s11227-024-06534-7 (2025).
- [15] Zhang, Y., Hua, W., Zhou, Z., Suh, G. E. and Delimitrou, C.: Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices, *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, pp. 167–181 (online), DOI: 10.1145/3445814.3446693 (2021).
- [16] Gan, Y., Liang, M., Dev, S., Lo, D. and Delimitrou, C.: Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices, *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, pp. 135–151 (online), DOI: 10.1145/3445814.3446700 (2021).
- [17] Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J. and Wu, Z.: MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems, *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 338–347 (online), DOI: 10.1109/ICSE-SEIP52600.2021.00043 (2021).
- [18] Wu, L., Tordsson, J., Elmroth, E. and Kao, O.: MicroRCA: Root Cause Localization of Performance Issues in Microservices, *NOMS 2020–2020 IEEE/IFIP Network Operations and Management Symposium*, IEEE, pp. 1–9 (online), DOI: 10.1109/NOMS47738.2020.9110353 (2020).
- [19] Colarusso, C., De Caro, A., Falco, I., Goglia, L. and Zimeo, E.: A Distributed Tracing Pipeline for Improving Locality Awareness of Microservices Applications, *Software: Practice and Experience*, Vol. 54, No. 6, pp. 1118–1140 (online), DOI: 10.1002/spe.3317 (2024).
- [20] Sampaio, A. R., Rubin, J., Beschastnikh, I. and Rosa, N. S.: Improving Microservice-Based Applications with Runtime Placement Adaptation, *Journal of Internet Services and Applications*, Vol. 10, No. 1, p. 4 (online), DOI: 10.1186/s13174-019-0104-0 (2019).
- [21] Adeppady, M., Chiasserini, C. F., Karl, H. and Giaccone, P.: iPlace: An Interference-Aware Clustering Algorithm for Microservice Placement, *Proceedings of the IEEE International Conference on Communications (ICC)*, Seoul, Republic of Korea, IEEE, pp. 5457–5462 (online), DOI: 10.1109/ICC45855.2022.9839222 (2022).
- [22] Zhu, J., Yang, R., Sun, X., Wo, T., Hu, C., Peng, H., Xiao, J., Zomaya, A. Y. and Xu, J.: QoS-Aware Co-Scheduling for Distributed Long-Running Applications on Shared Clusters, *IEEE Transactions on Parallel and Distributed Systems*, Vol. 33, No. 12, pp. 4818–4834 (online), DOI: 10.1109/TPDS.2022.3202493 (2022).
- [23] Shi, J., Zhang, H., Tong, Z., Chen, Q., Fu, K. and Guo, M.: Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters, *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, USENIX Association, pp. 403–417 (2023).
- [24] Kumar, N., Ruiz, P. M., Menon, V., Kabiljo, I., Pundir, M., Newell, A., Lee, D., Wang, L. and Tang, C.: Optimizing Resource Allocation in Hyperscale Datacenters: Scalability, Usability, and Experiences, *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, USENIX Association, pp. 507–528 (2024).
- [25] Denning, P. J. and Buzen, J. P.: The Operational Analysis of Queueing Network Models, *ACM Computing Surveys*, Vol. 10, No. 3, pp. 225–261 (online), DOI: 10.1145/356733.356735 (1978).
- [26] Tang, B., Luo, J., Obaidat, M. S. and Vijayakumar, P.: Container-Based Task Scheduling in Cloud-Edge Collaborative Environment Using Priority-Aware Greedy Strategy, *Cluster Computing*, Vol. 26, pp. 3689–3705 (online), DOI: 10.1007/s10586-022-03765-2 (2023).